ECO-MONITOR LAB ENERGY & ENVIRONMENT AUDITOR

A Smart IoT-Based Energy and Environmental Monitoring System for Sustainable Laboratory Management

College Name: Jain Deemed-to-be University
Department: School of Computer Science & Information Technology

Course / Semester: BCA 2nd Year, 4th Semester
Project Title: *Eco-Monitor Lab Energy & Environment Auditor*
Project Type: Python Mini Project

Submitted By:

1. Mohammed Rayyan Hussain-24BCAR0464
2. Harsh Vardhan-24BCAR0439
3. M Mithun-24BCAR0455

Submitted To: Prof. Prathima

Academic Year: 2026
Submission Date: 3/04/2026

CERTIFICATE

This is to certify that the mini project entitled **"Eco-Monitor Lab Energy & Environment Auditor"** is a bona fide work carried out by **Mohammed Rayyan Hussain, Harsh Vardhan, M Mithun** in partial fulfilment of the requirements for the award of the BCA degree. The project was completed under the guidance of **Prof. Prathima** and reflects the sincere effort, technical understanding, and teamwork of the students.

The work presented in this report is original to the best of our knowledge and has not been submitted elsewhere for the award of any degree or diploma.

Guide Signature: _____

Head of the Department: _____

Date: 3/04/2026

DECLARATION

We, **Mohammed Rayyan Hussain, Harsh Vardhan and M Mithun,** hereby declare that the project report titled **"Eco-Monitor Lab Energy & Environment Auditor"** is an original piece of work completed by us under the guidance of **Prof. Prathima**. This report has been prepared exclusively for academic submission as part of our BCA mini project requirement.

We further declare that this project has not been copied from any published source in a manner that would violate academic integrity. All references and supporting materials used in the preparation of this report have been duly acknowledged in the references section.

Date: 3/04/2026

Student Signature: _____

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide **Prof. Prathima,** for the constant support, valuable suggestions, and encouragement provided throughout the development of this project. Their guidance helped us understand the practical aspects of integrating hardware, software, and data visualisation into a single working system.

We also thank the Head of the Department and all faculty members of the BCA department for providing us with the opportunity and resources needed to complete this mini-project successfully. We are grateful to our friends and classmates for their cooperation, and to our families for their support and encouragement.

This project has helped us strengthen our skills in Python, IoT integration, database handling, backend development, and dashboard design. The experience has been both educational and motivating, and we sincerely appreciate everyone who contributed directly or indirectly to the completion of this work.

ABSTRACT

Energy efficiency and environmental monitoring are becoming increasingly important in educational institutions, especially in computer laboratories, where devices are left powered on for extended periods. The project **"Eco-Monitor Lab Energy & Environment Auditor"** is designed to monitor laboratory energy usage and environmental conditions in real time using an Arduino-based IoT system. The system collects data from sensors for temperature, humidity, and light intensity, while the control logic is structured to support current sensing and power estimation for energy auditing.

The sensor data is transmitted to a Python-based listener through serial communication, processed, and stored in a database for historical analysis. A Fast API-based backend exposes REST endpoints that a frontend dashboard can use to display live readings, usage trends, and calculated indicators, such as estimated energy consumption, cost, and carbon footprint. The system also supports relay-based device control, making it possible to remotely switch a load ON or OFF.

The project demonstrates the practical integration of embedded systems, Python programming, database management, and web technologies. It is relevant for sustainable lab management, energy conservation, and smart campus initiatives. The solution is modular, scalable, and suitable for educational and real-world monitoring environments.

Table of Contents

1. INTRODUCTION 8
1.1 Background 8
1.2 Problem Statement 8
1.3 Project Objective 8
1.4 Scope of the Project 9
2. PROBLEM STATEMENT 10
3. OBJECTIVES 11
4. SYSTEM ARCHITECTURE 12
4.1 Architecture Overview 12
4.2 Data Flow 12
4.3 Working Principle 12
5. HARDWARE COMPONENTS 13
5.1 Arduino Uno 13
5.2 DHT11 Sensor 13
5.3 LDR Sensor 13
5.4 Relay Module 13
5.5 Breadboard and Jumper Wires 13
5.6 USB Cable 13
5.7 Hardware Summary Table 14
6. SOFTWARE STACK 15
6.1 Arduino IDE 15
6.2 Python 15
6.3 FastAPI / Flask 15
6.4 Database 15
6.5 React 15
6.6 Visualisation Libraries 15
6.7 Software Summary Table 16

7. IMPLEMENTATION 17
7.1 Circuit Implementation 17
7.2 Data Acquisition 17
7.3 Serial Communication 17
7.4 Database Storage 17
7.5 Backend API Development 17
7.6 Dashboard Development 18
7.7 Energy, Cost, and Carbon Estimation 18
8. ALGORITHM / WORKFLOW 19
8.1 Workflow Steps 19
8.2 Flowchart Placeholder 19
10. RESULTS AND OBSERVATIONS 20
9.1 Sensor Readings 20
9.2 Dashboard Output 20
9.3 Relay Control Response 20
9.4 Historical Data Storage 20
9.5 Sample Observation Statement 20
10. IMPACT AND APPLICATIONS 21
10.1 Applications 21
10.2 Impact 21
11. LIMITATIONS 22
12. FUTURE ENHANCEMENTS 23
14. REFERENCES 25
15. APPENDIX 26
Appendix A: Pin Connections 26
Appendix B: Database Tables 26
Appendix C: Diagram & Screenshots 27

1. INTRODUCTION

1.1 Background

Laboratories in educational institutions often contain multiple electrical and electronic devices that remain active for long hours. In many cases, energy usage is not monitored closely, leading to unnecessary electricity consumption, increased operational cost, and reduced awareness of sustainability. In addition, the indoor environment of a laboratory—such as temperature, humidity, and lighting—can affect both comfort and efficient equipment usage.

The rapid growth of Internet of Things (IoT) technologies has made it possible to build low-cost smart monitoring systems that collect real-time data, store it digitally, and present it in a useful form for analysis. By combining embedded hardware with Python-based processing and web visualisation, it is possible to create a compact and practical energy and environment auditing system.

1.2 Problem Statement

Most laboratories do not have a simple and affordable system to monitor energy usage and environmental conditions in real time. As a result, lights, fans, and devices may remain ON even when they are not needed. This leads to unnecessary power consumption, higher electricity bills, and increased carbon footprint. There is a need for a practical monitoring solution that can help users observe usage patterns, estimate cost, and support energy conservation.

1.3 Project Objective

The main objective of this project is to design and implement an IoT-based laboratory monitoring system that:

- measures environmental conditions such as temperature, humidity, and light intensity,
- supports energy monitoring and power estimation,
- stores and processes sensor data using Python,
- displays live readings on a dashboard,
- estimates electricity cost and carbon footprint,
- and enables remote switching of devices through a relay module.

1.4 Scope of the Project

This project is intended for small-scale laboratory monitoring and demonstration purposes. It is suitable for educational environments and can be extended later for larger smart campus applications. The current version focuses on data collection, real-time display, historical logging, and control integration. The architecture is designed to support future expansion with current sensing hardware, cloud storage, and mobile access.

2. PROBLEM STATEMENT

Energy wastage in laboratories is a common issue due to the absence of real-time monitoring and control. Devices may stay powered unnecessarily, and environmental conditions may not be observed continuously. Manual checking is inefficient and often does not provide historical data for analysis.

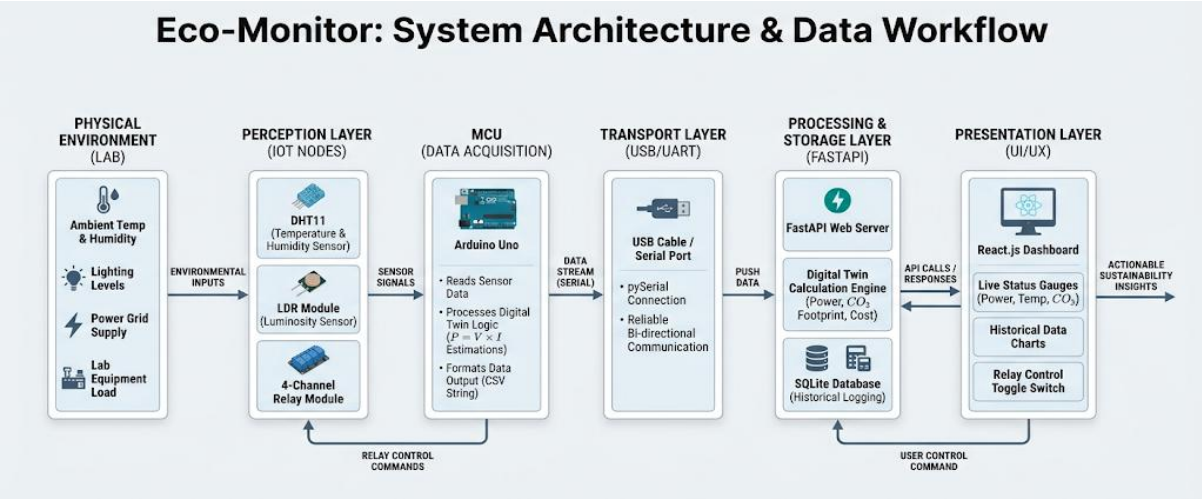
This project addresses the problem by creating a smart monitoring system that continuously observes laboratory conditions, estimates energy-related impact, and provides a digital interface for better decision-making. The system aims to improve awareness, reduce wastage, and demonstrate how IoT can support sustainable energy management.

3. OBJECTIVES

The project was developed with the following objectives:

- To monitor laboratory temperature, humidity, and light conditions in real time.
- To support electrical energy monitoring and power estimation.
- To store sensor data systematically for later analysis.
- To provide a live dashboard for easy observation.
- To enable relay-based device control from the software interface.
- To calculate the estimated cost and carbon footprint based on power usage.
- To demonstrate the integration of Arduino, Python, backend APIs, and frontend visualisation.

4. SYSTEM ARCHITECTURE



4.1 Architecture Overview

The system follows a layered architecture. Sensor data is collected by the Arduino Uno from the connected hardware modules. The Arduino sends the data to a Python listener through USB serial communication. The Python layer processes the readings, stores them in a database, and performs energy-related calculations. A backend API exposes the processed data to the frontend dashboard, where users can view live readings, graphs, and device control options.

4.2 Data Flow

Sensors → Arduino Uno → Serial Communication → Python Listener → Database → FastAPI Backend → React Dashboard → User

4.3 Working Principle

1. Sensors measure environmental data and send values to the microcontroller.
2. Arduino formats the readings into structured output.
3. Python reads the incoming serial data and stores it in the database.
4. The backend processes current and historical records.
5. The frontend displays live and historical information in a visual form.
6. The relay module can be controlled from software to switch a device ON or OFF.

5. HARDWARE COMPONENTS

5.1 Arduino Uno

Arduino Uno acts as the central microcontroller of the system. It reads inputs from the sensors, processes the values, and transmits them to the Python application using serial communication. It also controls the relay module for switching connected devices.

5.2 DHT11 Sensor

The DHT11 sensor is used to measure temperature and humidity. These parameters are important for understanding the internal environment of the laboratory and for identifying uncomfortable or inefficient operating conditions.

5.3 LDR Sensor

The Light Dependent Resistor (LDR) detects light intensity. It helps identify whether lights are left on unnecessarily and supports environmental awareness within the laboratory.

5.4 Relay Module

The relay module allows software-based ON/OFF control of connected electrical devices. It adds a control feature to the monitoring system and improves interactivity.

5.5 Breadboard and Jumper Wires

The breadboard and jumper wires are used for prototyping the circuit without soldering. They allow quick testing, debugging, and hardware modification during development.

5.6 USB Cable

The USB cable provides power to the Arduino and enables serial communication with the computer system running Python.

5.7 Hardware Summary Table

Component	Purpose	Role in Project
Arduino Uno	Main controller	Reads sensors and sends data
DHT11	Temp/Humidity sensing	Environment monitoring
LDR	Light sensing	Light usage detection
Relay Module	Device control	Switches load ON/OFF
Breadboard	Prototyping	Easy circuit assembly
Jumper Wires	Connections	Hardware interfacing
USB Cable	Communication	Arduino-to-PC data transfer

6. SOFTWARE STACK

6.1 Arduino IDE

Arduino IDE is used to write and upload the microcontroller code. It is also used for serial monitoring and initial debugging.

6.2 Python

Python is the main processing language used in the project. It handles serial communication, data parsing, energy estimation logic, database insertion, and backend operations.

6.3 FastAPI / Flask

A Python-based backend framework is used to expose REST APIs. These APIs provide current status, history, summary, and relay control endpoints to the frontend application.

6.4 Database

SQLite is used for storing sensor logs, summaries, and configuration values. It is lightweight, easy to manage, and suitable for a mini project.

6.5 React

React is used to design the interactive dashboard interface. It helps present live readings, charts, and action buttons in a clean and responsive layout.

6.6 Visualisation Libraries

Plotly, Matplotlib, or Chart.js can be used to generate charts for power usage, temperature variation, and historical analysis.

6.7 Software Summary Table

Tool / Library	Purpose
Arduino IDE	Microcontroller programming
Python	Data processing and backend logic
pySerial	Serial communication
FastAPI / Flask	REST API development
SQLite	Local data storage
React	Dashboard UI
Plotly / Chart.js	Graphical visualization

7. IMPLEMENTATION

7.1 Circuit Implementation

The hardware components were connected to the Arduino Uno using a breadboard and jumper wires. The DHT11 sensor was connected for temperature and humidity monitoring, the LDR was used as a light sensor, and the relay module was connected to enable remote switching. The circuit was arranged in a modular form so that future components can be added easily.

7.2 Data Acquisition

The Arduino reads sensor values at regular intervals and sends them in a structured CSV format through the USB serial connection. This approach simplifies parsing on the Python side and ensures consistent communication.

7.3 Serial Communication

Python uses the pySerial library to listen to the Arduino output. Each line of data is read, parsed, and validated before being stored in the database. This creates a continuous pipeline from hardware to software.

7.4 Database Storage

Readings are inserted into a database so that historical analysis can be performed. The database stores timestamped sensor values, relay state, and calculated metrics such as estimated power and energy.

7.5 Backend API Development

The backend provides REST endpoints that supply current status, historical records, summaries, and control actions. The API structure allows easy integration with any frontend application.

7.6 Dashboard Development

The React dashboard presents live readings, graphs, and control options. The interface is designed to be simple, responsive, and suitable for demonstration during a presentation or viva.

7.7 Energy, Cost, and Carbon Estimation

The project includes a calculation layer for estimating power, energy, electricity cost, and carbon footprint. These values are useful for sustainability awareness and lab management.

Formula Summary

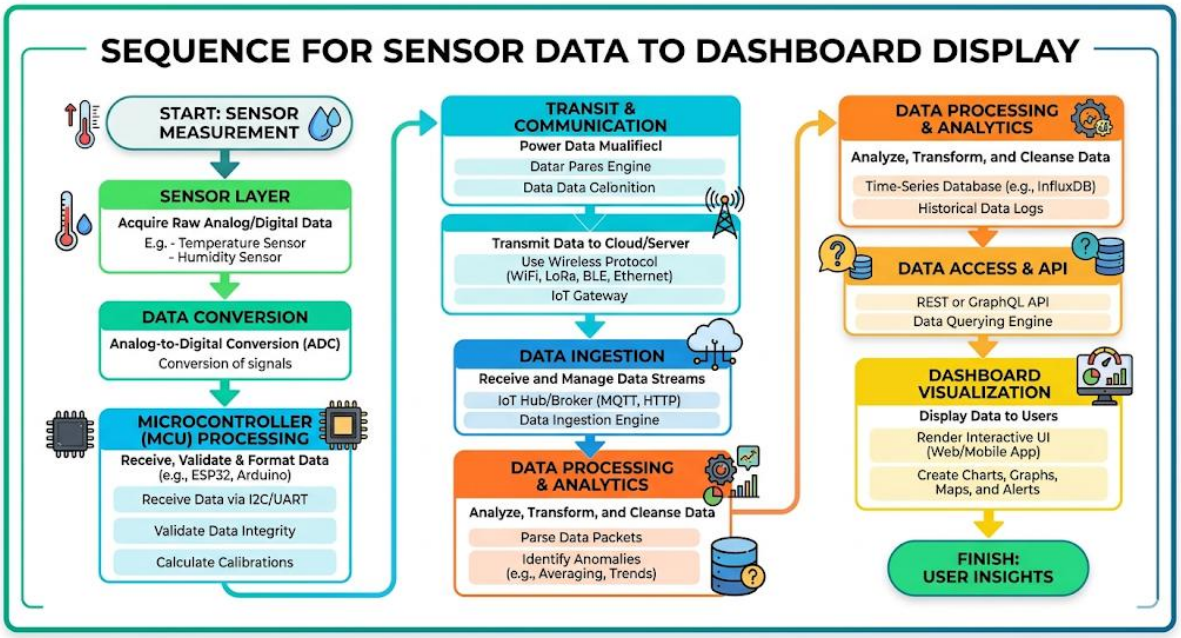
- **Power (W) = Voltage × Current**
- **Energy (kWh) = Power × Time / 1000**
- **Cost = Energy × Electricity Rate**
- **Carbon Footprint = Energy × Carbon Factor**

8. ALGORITHM / WORKFLOW

8.1 Workflow Steps

1. Initialize Arduino, sensors, and relay module.
2. Read environmental and electrical values from the connected components.
3. Format sensor readings into CSV structure.
4. Send data to Python through serial communication.
5. Parse, validate, and store the readings in the database.
6. Compute power, energy, cost, and carbon estimates.
7. Expose the processed data through REST APIs.
8. Display live and historical results on the dashboard.
9. Accept relay ON/OFF commands from the user interface.

8.2 Flowchart Placeholder



10. RESULTS AND OBSERVATIONS

9.1 Sensor Readings

The system successfully captures temperature, humidity, and light-related values from the connected sensors. The readings vary depending on environmental changes and sensor placement, which confirms that the system is responsive and functional.

9.2 Dashboard Output

The dashboard displays live values in a visual format, making it easy to understand the state of the laboratory environment at a glance. Graphs and indicators help users identify patterns and potential wastage.

9.3 Relay Control Response

The relay module responds to control commands from the software interface, demonstrating the system's ability to perform remote switching.

9.4 Historical Data Storage

The stored records can be used to generate trends and summary statistics such as mean, variance, and standard deviation for power or temperature values. This supports analysis and reporting.

9.5 Sample Observation Statement

During testing, the system showed clear variation in sensor values when lighting conditions changed or when the environment around the sensors was modified. The dashboard reflected these changes in real time, confirming that the monitoring pipeline was working correctly.

10. IMPACT AND APPLICATIONS

The Eco-Monitor system has practical value in educational institutions where efficient energy usage and environmental awareness are important. The project can support smart lab management by identifying unnecessary energy usage and by providing visibility into room conditions.

10.1 Applications

- Computer laboratories
- Smart classrooms
- Hostel rooms
- Faculty rooms
- Campus energy audits
- Sustainability projects

10.2 Impact

- Helps reduce energy wastage
- Encourages responsible device usage
- Improves awareness of laboratory conditions
- Supports sustainability and green campus goals
- Demonstrates the usefulness of low-cost IoT solutions

This project shows how embedded systems and full-stack development can be used together to create a useful, scalable, and socially relevant application.

11. LIMITATIONS

Although the project is functional and practical, it has certain limitations:

- Accurate power measurement depends on a dedicated current sensor.
- Sensor readings may require calibration for improved precision.
- Simplified calculations may assume standard voltage and load behaviour.
- Long-term deployment would require enclosure, power safety, and stronger hardware protection.

These limitations do not reduce the educational value of the project; instead, they highlight realistic engineering conditions and possible future improvements.

12. FUTURE ENHANCEMENTS

The system can be expanded in several ways:

- Integration of a real current sensor, such as ACS712 or INA219
- Cloud deployment for remote monitoring
- Mobile-friendly app support
- User authentication and role-based access
- Automated energy-saving rules
- Multi-room monitoring support
- AI-based anomaly detection for unusual usage patterns
- CSV export and email/Telegram alerts

These enhancements can transform the mini project into a more advanced smart campus solution.

13. CONCLUSION

The project **“Eco-Monitor Lab Energy & Environment Auditor”** successfully demonstrates a practical IoT-based approach to laboratory energy and environmental monitoring. By combining Arduino hardware, Python-based processing, a database, and a web dashboard, the system provides a complete framework for observing laboratory conditions and supporting energy awareness.

The project helped us strengthen our understanding of embedded systems, serial communication, backend APIs, database integration, and dashboard design. It also reflects an important real-world idea: technology can be used not only for convenience, but also for sustainability and responsible resource usage.

Overall, this mini project is academically useful, technically relevant, and suitable for practical demonstration. It can also be expanded into a more advanced monitoring solution for larger educational environments in the future.

14. REFERENCES

1. Arduino Official Documentation, https://www.arduino.cc/
2. Python Official Documentation, https://www.python.org/
3. FastAPI Documentation, https://fastapi.tiangolo.com/
4. SQLite Documentation, https://www.sqlite.org/docs.html
5. React Documentation, https://react.dev/
6. DHT11 Sensor Datasheet / Module Reference
7. Relay Module and LDR Module Reference Materials
8. pySerial Documentation

15. APPENDIX

Appendix A: Pin Connections

Component	Arduino Pin
DHT11 Data	D2
LDR Output	A1
Relay Input	D8
VCC	5V
GND	GND

Appendix B: Database Tables

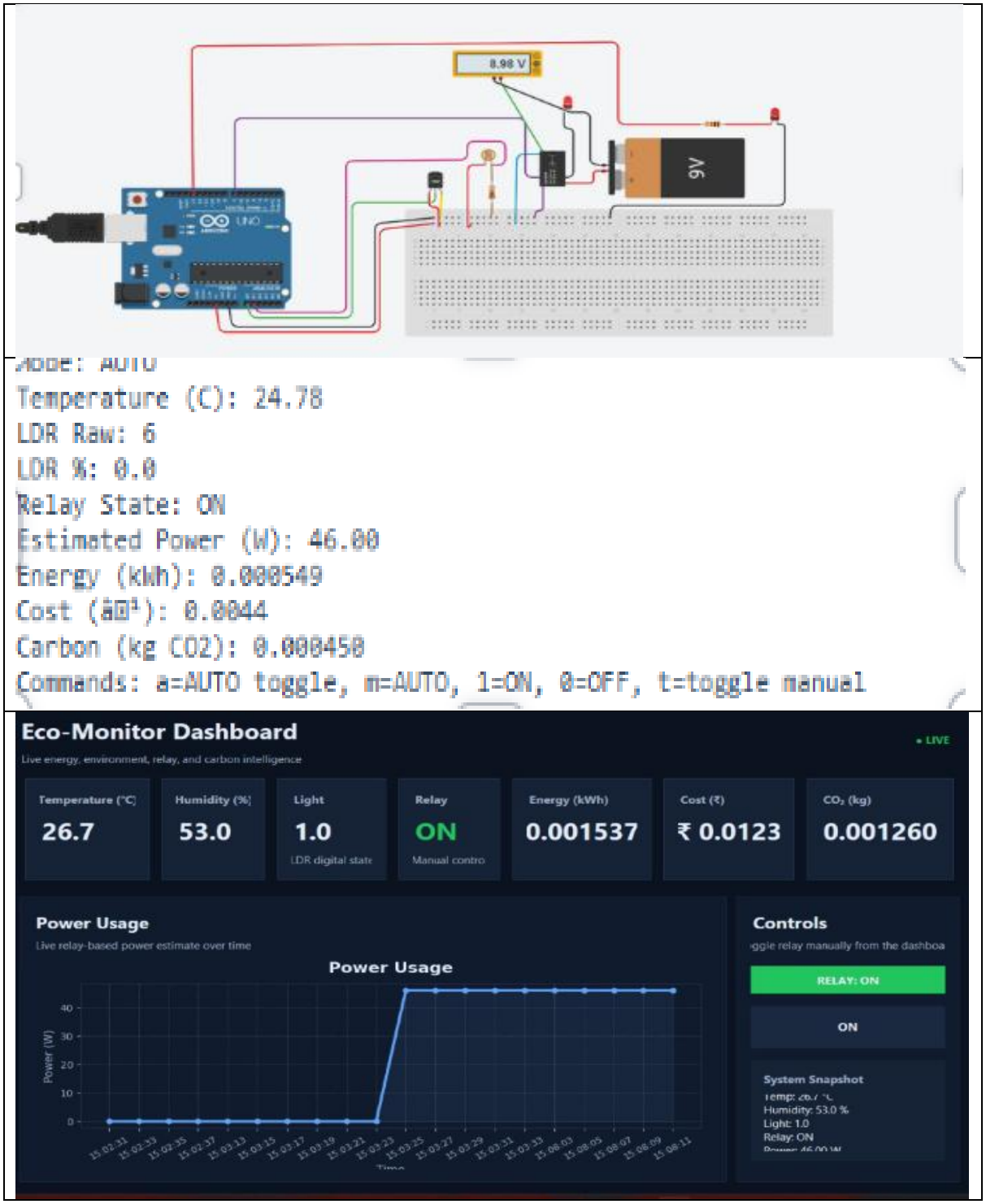
readings

- id
- timestamp
- current value
- temperature
- humidity
- light level
- relay state
- power
- energy

events

- event id
- timestamp
- event type
- message

settings

- key
- value

Appendix C: Diagram & Screenshots